

Recursive Execution Decomposition (RED): A Verification-Gated Protocol for Surfacing Hidden Assumptions and Missing Fundamentals

Abstract

We introduce **Recursive Execution Decomposition (RED)**, a verification-gated protocol for auditing plans, systems, and research methods by recursively decomposing them into executable actions until each leaf maps to a registered primitive. A primitive is considered valid only if three conditions can be verified: the required **tool exists**, the necessary **knowledge exists**, and appropriate **access exists**.

RED enforces a strict separation between design requirements and current-state reality: any dependency that cannot be reproducibly verified is treated as **missing work**, not as an assumption. The method produces auditable artifacts—hierarchical decomposition tables and primitive-level audits—that convert hidden execution dependencies into an actionable backlog of resolution tasks.

We demonstrate RED across multiple case studies, including neural architecture analysis (Transformers), engineering feasibility audits (Continuous Thought Machines), tool-call interface failures, and post-failure debugging of agentic systems. In each case, RED surfaces operational constraints, unstated assumptions, and missing measurements that are not evident in high-level descriptions. We discuss limitations, failure modes, and safeguards, and position RED as a practical methodology for human–AI collaboration, research-to-deployment translation, and systematic root-cause analysis.

1. Introduction

High-level descriptions and benchmark results often hide the execution realities that determine whether an idea succeeds in practice. A plan can be logically coherent yet fail immediately because the execution substrate (tools, environment, permissions, operational ownership) does not match implicit assumptions. These failures frequently surface late—after time has been spent coding, debugging, or attempting reproduction.

This work originated from collaboration with LLM-based coding agents as a non-expert. The models often produced confident, plausible plans that implicitly assumed prerequisites (“the table exists,” “the tool is available,” “the environment is configured”). Execution failures revealed that the plan had conflated intended design with current reality.

RED is motivated by a narrow goal: not “smarter plans,” but a systematic way to convert implicit execution assumptions into explicit, auditable objects.

1.1 Motivating example (execution substrate mismatch)

In a role-based workflow layered on top of Git, an LLM suggested that multiple agents could act in the same working directory without Git worktrees—a recommendation corroborated by other models when consulted. The approach was conceptually plausible but failed immediately: Git operations impose a hard coupling between branch state and working directory state, and concurrent branch switching in the same directory is unsafe. The error was not “bad reasoning” so much as a missing constraint about the substrate.

This kind of failure—where the substrate constraint is unknown, not just unmet—motivated the knowledge/skill audit as a mandatory component of RED—a methodology that forces substrate constraints to be stated and verified early.

1.2 Problem: missing fundamentals

Many failures are not conceptual errors but missing fundamentals:

- a tool/library does not exist in the target environment,
- a required command/invoke is unknown,
- access is missing (credentials, permissions, environment constraints),
- the plan omits who owns an operation (migration, deployment, secrets),
- the plan assumes properties that are not verified.
- the LLM generates an incomplete plan, but non-experts cannot verify completeness.

1.3 Contributions

This paper presents **Recursive Execution Decomposition (RED)**:

1. A repeatable protocol for decomposition with a **primitive registry** as a stop condition.
2. A strict separation between **design requirements** and **current-state reality**.
3. A verification gate: **unverifiable = missing**, producing resolution tasks.
4. Standard artifacts (tables) that make analysis inspectable by non-experts and experts.
5. Multiple case studies demonstrating RED outputs as constraints/assumptions/gaps (hypotheses).

1.4 Why RED is not “just a checklist”

Checklists enumerate items; RED defines a **recursion rule and a stop condition**. Its key mechanism is not “having a list,” but enforcing:

- decomposition until primitive-level operations,
- explicit mapping of actions to resources,
- and a binary verification policy that forbids marking items as “assumed okay.”

In practice, LLMs tend to stop early and gloss over details unless driven one action at a time. RED's structure and gates exist to prevent that premature abstraction collapse.

1.5 What RED is not

To clarify scope and prevent misapplication:

- **Not primarily a planner:** RED's strength is verification, not generation. While decomposing goals can produce plan-like structures, RED's value is in surfacing gaps, not in creative planning.
- **Not an optimizer:** RED surfaces what is missing; it does not select the best path.
- **Not a proof of correctness:** RED identifies unverified dependencies; it does not guarantee the plan will succeed.
- **Not exhaustive:** RED is bounded by registry quality and decomposition depth; unknown unknowns may remain.
- **Not automatic:** RED requires human judgment for decomposition quality and verification decisions.

RED's value is converting implicit assumptions into explicit, auditable objects—not replacing expertise or guaranteeing outcomes.

2. Background & Related Work (positioning)

RED sits at the intersection of hierarchical decomposition, engineering verification, and human–AI collaboration. Unlike classical planning that assumes a fixed action model (Fikes & Nilsson, 1971; McDermott et al., 1998), RED treats the action model itself (tools, access, invocation knowledge) as part of what must be audited.

- **Hierarchical decomposition:** RED resembles hierarchical task networks (Sacerdoti, 1974; Ghallab et al., 2004) but differs in its recursion stop condition (primitive registry) and mandatory verification audits at each leaf.
- **Engineering preflight checks:** RED generalizes preflight dependency checks and Failure Mode and Effect Analysis (Stamatis, 2003) by requiring reproducible verification evidence for each dependency.
- **Failure analysis:** RED is complementary to postmortems; it aims to surface missing fundamentals *before* implementation, similar to how checklists prevent omission errors

(Gawande, 2010), but with recursive structure rather than flat enumeration.

- **Human–AI collaboration:** RED constrains LLM outputs into structures that support validation, complementing chain-of-thought prompting (Wei et al., 2022) and agentic reasoning patterns (Yao et al., 2022) with verification gates that reduce silent assumptions.

Key distinctions: Unlike HTN planning, RED does not assume a complete and correct action model; the action model itself (tools, access, invocation knowledge) is subject to audit. Unlike FMEA and checklists, RED enforces recursive decomposition to a stop condition (primitive registry) and applies a binary verification gate in which unverifiable dependencies are explicitly treated as missing work items rather than assumptions. RED's contribution is not novel decomposition, but the combination of recursive structure, primitive-grounded stopping, and strict verification policy.

This section provides positioning rather than an exhaustive survey.

3. RED: Recursive Execution Decomposition (Method)

3.1 Definitions

- **Artifact:** a plan, architecture, method section, or workflow to audit.
- **Action:** an operational step required to execute the artifact.
- **Decomposition:** mapping an action into sub-actions.
- **Primitive Registry (P):** curated set of known primitives used as stop condition.
- **Primitive:** an action $p \in P$ that is atomic only when it satisfies:
 1. **tool exists**, 2) **knowledge exists**, 3) **access exists**.
- **Design requirement:** a dependency implied by the plan.
- **Current-state reality:** what is verified present now.
- **VERIFIED_HAVE / MISSING:** binary statuses; VERIFIED_HAVE requires reproducible evidence.
- **Missing fundamentals:** the set of MISSING dependencies converted into tasks.

3.1.1 Example primitive registry entries (software engineering)

Examples of primitives used in this paper's reproducible registry:

- `read_file(path)`
- `list_files(path)`
- `search_files(query)`
- `query_db(sql)`

- `run_tests(pattern)`
- `apply_migration(name)`

Different domains can define different registries.

3.1.2 Registry governance

A natural question: *Who defines the primitive registry, and how stable is it?*

Ownership: The registry can be curated by an individual, team, or organization. For personal use, a single practitioner maintains their own registry. For team use, a shared registry provides consistency across analyses.

Evolution: Registries are not static. When RED reveals a missing primitive (an action that should be atomic but lacks tool/knowledge/access), the failure becomes a candidate for registry expansion. Post-failure refinement is the primary growth mechanism: failures teach which primitives were missing.

Versioning: Different registry versions produce different RED outputs. $RED(\text{artifact}, P_1)$ may surface different gaps than $RED(\text{artifact}, P_2)$. This is a feature, not a bug—it makes explicit that analysis quality depends on registry maturity.

Key claim: RED is only as good as its registry, but failures systematically improve it. A registry that has survived many failure-driven refinements is more trustworthy than one created from first principles.

3.2 Protocol overview

RED is a recursive expansion procedure with mandatory output artifacts and audits.

Step A — Scope lock

Define:

- artifact under analysis,
- primary goal/claim,
- success criteria,
- and what is explicitly out-of-scope.

Step B — Recursive decomposition until primitives

Conceptual procedure:

1. Initialize queue/stack Q with the top-level action(s).
2. While Q not empty:

- pop a
- generate children decompose(a, K)
- record parent→child mapping
- if child is primitive: keep as leaf
- else push child into q

3. Stop when all leaves are primitives or safety gates trigger.

Step C — Expanded decomposition tables (mandatory)

At minimum, produce L1→L2 parent→child mapping with columns:

- Parent action
- Child action
- Resources touched (tools/services)
- Resources required (inputs)
- Outputs produced
- Primitive? (Y/N)
- Status (VERIFIED_HAVE / MISSING)

Step D — Design vs current-state audit (anti-hallucination rule)

RED separates:

- “Design requires X” vs
- “X is verified present now”

If design requires X but presence is not verified, X becomes MISSING.

Step E — Primitive-level audits (mandatory at stop)

For each leaf primitive:

1. **Tools audit:** tool/library/service exists and is accessible.
2. **Inputs audit:** required inputs exist and are available.
3. **Outputs audit:** outputs are defined and consumed by some downstream step.
4. **Knowledge/skill audit:** required know-how exists; missing knowledge becomes a task.

Step F — Missing fundamentals become tasks

Every MISSING dependency yields a concrete resolution task.

3.3 Verification policy (VERIFIED_HAVE is strict)

VERIFIED_HAVE requires a reproducible verification method (command output, code reference, config evidence). Otherwise treat as MISSING.

Examples of acceptable evidence:

- *Tool exists*: successful execution of `pip show library` or a minimal import script
- *Knowledge exists*: documented procedure or demonstrated execution by a team member
- *Access exists*: successful authenticated API call or permission check output

Vague claims ("I think we have this") do not qualify. If evidence cannot be produced on demand, the dependency is MISSING.

3.4 Safety gates

RED may stop early if:

- max depth exceeded,
- max fan-out exceeded.

These gates prevent unbounded analysis and force scoping decisions.

3.5 Non-expert vs expert-guided modes

- **Non-expert mode**: structure + binary verification reduces silent assumptions.
- **Expert-guided mode**: experts refine registries, validate completeness, and prioritize high-impact gaps.

3.5.1 AI-assisted RED

RED is well-suited to human–AI collaboration. LLMs can assist with several RED activities while humans retain judgment and verification authority.

Where LLMs assist effectively:

- *Decomposition generation*: Given an action, LLMs can propose sub-actions. The human validates whether the decomposition is complete and correct.
- *Primitive suggestion*: LLMs can suggest which actions might be primitives and what tools/knowledge they require.
- *Audit table drafting*: LLMs can generate initial audit tables (tools, inputs, outputs, knowledge) that humans then verify and correct.
- *Gap identification*: LLMs can flag potential missing dependencies based on pattern recognition across the decomposition.
- *Mitigation planning*: For each identified gap, LLMs can propose resolution strategies and concrete mitigation steps.

- *Plain-language explanation:* LLMs can translate technical constraints and gaps into accessible language, enabling non-experts to understand and validate findings they could not have articulated themselves.

Where humans remain essential:

- *Verification decisions:* Only humans can confirm whether evidence is sufficient for VERIFIED_HAVE. LLMs may hallucinate that dependencies exist.
- *Registry governance:* Humans decide which primitives belong in the registry and how to evolve it.
- *Judgment calls:* Prioritization, scoping, and "is this decomposition actually useful?" require human judgment.
- *Ground truth:* Humans must execute verification commands, check file existence, and confirm access.

Practical pattern: The most effective workflow is LLM-generates, human-validates. The LLM proposes decompositions and audits; the human challenges, corrects, and verifies. This combines LLM breadth with human judgment, and RED's structure ensures that LLM hallucinations are caught at verification gates rather than silently assumed.

The case studies in this paper were developed using this AI-assisted pattern: RED decomposition with LLM assistance, followed by human validation against known constraints and observable system behavior.

3.6 Minimal formalization

Let A be actions, $P \subset A$ primitives, K decomposition context (domain knowledge used to generate sub-actions).

```
RED(a, K) =
  if a ∈ P then {a}
  else U_{a' ∈ decompose(a, K)} RED(a', K)
```

Let S be current system state and $\text{audit}(p, S)$ produce dependency claims labeled VERIFIED_HAVE or MISSING.

```
LeafPrimitives = RED(a, K)
AuditReport = { audit(p, S) | p ∈ LeafPrimitives }
MissingFundamentals = { d ∈ AuditReport | status(d) = MISSING }
```

4. Case Study A — Transformer (Attention Is All You Need)

4.1 Setup

We apply RED to Vaswani et al. (2017) to demonstrate how RED converts a clean conceptual description into an execution-oriented checklist of constraints, assumptions, and gaps. The objective is not to criticize the Transformer but to show the protocol.

Methodology note: The following constraints, assumptions, and gaps were identified through RED decomposition with LLM assistance, then validated against known limitations discussed in ML literature. This demonstrates RED's value for human–AI collaboration: the methodology surfaces findings that match expert consensus without requiring the human operator to have prior domain expertise.

Method-centric framing: These findings are not presented as model critiques, but as artifacts produced by applying the RED protocol. They demonstrate how the method operationalizes abstract architectural claims into verifiable constraints and testable gaps.

Epistemic labeling:

- **Constraints:** follow from stated primitives and resource scaling.
- **Assumptions:** premises required for the story to hold.
- **Gaps:** missing primitives or missing measurements (testable hypotheses).

4.2 Constraints (identified by RED)

Memory wall: attention requires an $N \times N$ matrix $\rightarrow$ quadratic activation scaling.

Latency floor: stacked layers impose sequential dependencies across depth.

Bandwidth tax: normalization/residuals are often bandwidth-bound.

Positional resolution issues: certain encodings have extrapolation limits.

4.3 Assumptions (surfaced by RED)

- training distribution approximates deployment,
- loss proxy aligns with correctness,
- order is injected externally (positional encoding),
- verification is external (not native to the architecture).

4.4 Gaps / missing primitives (identified by RED)

- no persistent memory across context windows,
- no adaptive thinking time (fixed compute),

- no native grounding/verification.

4.5 Synthesized requirements (derived)

From constraints/gaps, common post-transformer requirements include:

- reduce long-context cost,
 - add persistent memory,
 - add adaptive compute,
 - integrate verification and grounding.
-

5. Case Study B — Continuous Thought Machines (CTM)

5.1 Setup

We apply RED to Darlow et al. (2025) as an engineering-feasibility audit. CTM maps well to RED because it makes operational claims (adaptive compute, interpretability narratives, synchronization representations) that correspond to measurable execution properties.

Methodology note: As with Case Study A, these findings were identified through RED decomposition with LLM assistance. The constraints and gaps align with engineering considerations that would apply to any recurrent architecture with similar claims.

Method-centric framing: These findings are not presented as critiques of CTM, but as artifacts produced by applying RED to its operational claims. The method converts conceptual descriptions into concrete deployment checklists.

5.2 Constraints (identified by RED)

Sequentiality wall: internal ticks impose $O(T)$ sequential dependencies; BPTT increases cost.

Batching/efficiency risk: halting reduces ticks but may not reduce wall-clock time under real batching.

Scaling pressure: neuron-level private models can increase parameter scaling and optimization complexity.

5.3 Assumptions (surfaced by RED)

Certainty-as-correctness: entropy/certainty correlates with correctness.

Interpretability narrative risk: “looking around” visualizations can be suggestive without causal intervention.

5.4 Gaps / missing measurements (identified by RED)

- measure throughput/latency under realistic batching and halting distributions,
- ablate backbones/resolutions to separate “looking” from feature extractor artifacts,
- measure stability and seed variance for reproducibility.

5.5 Summary

RED turns CTM’s conceptual claims into a concrete deployment checklist: what to measure, what to ablate, what to validate.

6. Limitations, Failure Modes, Safeguards

6.1 Limitations

- coverage bounded by primitive registry quality,
- RED cannot guarantee discovery of all unknown unknowns,
- recursion cost grows with depth/fan-out,
- strict binary verification can be conservative.

6.2 Failure modes

- plausible but incorrect decompositions,
- premature stopping,
- verification theater,
- scope creep.

6.3 Safeguards

- scope lock,
 - max depth/fan-out gates,
 - binary verification (no “assumed” state),
 - missing fundamentals must become tasks,
 - expert review to refine registry and catch missing branches.
-

7. Discussion: From Method to Practice

7.1 Where RED is useful

- engineering design reviews and implementation planning,
- research-to-deployment translation,
- human–AI collaboration where plans drift into assumptions,
- post-failure debugging and root cause analysis (see Section 7.4).

7.2 Future work: failure-driven refinement

Failures can refine registries and decomposition templates over time. A formal evaluation of this learning loop is future work.

7.3 Worked Example: Preventing Tool-Call Failure in Long, Symbol-Heavy Writes (`write_file`)

This worked example demonstrates RED as an *execution-feasibility* method rather than an abstract decomposition exercise. The artifact analyzed is a common LLM-agent workflow: writing a long Markdown document to disk using a tool-call interface.

7.3.1 Setup (artifact, goal, and failure mode)

Artifact: An agent plan to generate and write a long Markdown document (multi-section, code blocks, tables, heavy punctuation) via a file-writing tool.

Goal: Produce a correct final file on disk.

Execution substrate constraint: The tool interface requires the agent to emit a structured call (e.g., JSON function arguments). When the content becomes sufficiently long and symbol-heavy, the tool-call payload can become malformed (e.g., unescaped characters, broken delimiters, truncated structural braces). The failure is not “content too large” per se; it is that **the serialization boundary fails**—the function-call wrapper becomes invalid.

Observed failure (baseline / non-RED): The agent attempts to generate the entire document in one tool call. The model produces text whose structure conflicts with JSON serialization or exceeds the model’s reliability for maintaining valid structure under heavy punctuation. The result is a tool-call parse failure: the system rejects the call and the write does not occur.

Why this matters: This is a concrete case where a plan can be logically correct (“write the report”) but infeasible due to execution-interface constraints. RED’s purpose is to surface such constraints early and convert them into explicit mitigation tasks.

7.3.2 Baseline (non-RED) plan and why it fails

A typical baseline plan is:

1. Generate full Markdown document

2. Call `write_file(path, content)` with the entire document

This baseline implicitly assumes:

- the tool-call wrapper remains valid regardless of content complexity,
- the model can reliably serialize long symbol-heavy text into a structured argument format,
- the system can recover from serialization failure without losing work.

In practice, these assumptions are frequently false under real tool-call constraints. Importantly, the baseline plan fails *silently at the boundary*: the document generation may be correct, but the tool call is rejected.

7.3.3 RED application (Scope lock → primitives → audits)

We apply RED to the artifact “write a long Markdown file via tool call.”

Step A — Scope lock

- **In scope:** feasibility of completing the write using the tool-call substrate.
- **Out of scope:** content quality of the report itself; we focus on whether the write can be executed reliably.
- **Success criteria:** file is written to disk with complete content and tool-call execution is valid.

Step B — Decomposition to primitives

We decompose the top-level action:

- **A0:** “Write the report to disk”

into sub-actions:

- **A1:** “Generate report structure (outline)”
- **A2:** “Generate report section content”
- **A3:** “Prepare safe tool-call payloads”
- **A4:** “Write content to file (tool execution)”
- **A5:** “Verify written file matches expected content”

RED continues decomposition until leaf steps map to primitives. Using a software-engineering registry, the relevant primitives include:

- `write_file(path, content_chunk)`
- `read_file(path)`
- (optional) `hash(text)` / `diff(text_a, text_b)` as verification primitives

Step C — Expanded tables (illustrative branch)

Below is an illustrative parent→child mapping for the critical execution branch (serialization + write):

Tree recap:

```
A0 Write report to disk
  A3 Prepare safe tool-call payloads
    A3.1 Chunk content into safe units
    A3.2 Escape/validate each chunk for tool-call safety
  A4 Write each chunk via tool (primitive, repeated)
  A5 Verify written file (primitive)
```

Execution-order view (separate table)

The execution-order view shows the temporal sequence of the critical path, while the parent→child table below shows the hierarchical decomposition structure.

Step	Action
1	A3.1 Chunk content
2	A3.2 Escape/validate chunk
3	A4 Write chunk (repeat for each chunk)
4	A5 Verify written file

Parent action	Child action	Resources touched	Inputs required	Outputs produced	Primitive?	Status
A0 Write report to disk	A3 Prepare safe tool-call payloads	LLM tool-call wrapper / schema	Markdown content	chunk plan + validated chunks	No	MISSING missing)
A3 Prepare safe tool-call payloads	A3.1 Chunk content into safe units	none (planning)	full content	chunk list	No	VERIFIED
A3 Prepare safe tool-call payloads	A3.2 Escape/validate each chunk for tool-call safety	tool-call schema	chunk	validated chunk	No	MISSING

Parent action	Child action	Resources touched	Inputs required	Outputs produced	Primitive?	Status
A0 Write report to disk	A4 Write each chunk via tool	write_file	path + validated chunk	file updated	Yes	VERIFIED
A0 Write report to disk	A5 Verify written file	read_file (+ optional diff/hash)	path	file contents	Yes	VERIFIED

- A4 is executed once per chunk; A5 is executed after all writes (or periodically, depending on policy).

This table makes explicit a leaf that baseline plans omit: **serialization is a real execution dependency** and must be treated as a first-class step.

Step D — Design vs current-state audit

RED forces us to separate:

- **Design requires:** a reliable way to transmit long symbol-heavy content through the tool-call interface.
- **Current state:** tool-call interface is strict and fragile under long symbol-heavy strings.

Therefore the dependency “Prepare safe tool-call payloads” must be treated as **MISSING** unless verified.

7.3.4 Primitive-level audits and the missing fundamentals backlog

At the primitive level, RED audits the dependencies required for successful execution.

Tools audit:

- `write_file` exists and is callable (VERIFIED_HAVE if test call succeeds).
- `read_file` exists and is callable (VERIFIED_HAVE if test call succeeds).

Inputs audit:

- the report content exists (VERIFIED_HAVE if generated).
- the tool-call wrapper can safely carry the content (MISSING unless verified for worst-case content).

Outputs audit:

- output is a file on disk.
- verify output is consumed by the next step (read-back check).

From these audits, RED generates a missing-fundamentals backlog. Example tasks:

- **T1:** Add a chunking strategy for symbol-heavy content (e.g., section-by-section, or fixed-size chunks).
- **T2:** Add write-session protocol (buffering + finalize) to avoid requiring one-shot serialization.
- **T3:** Add verification step: read-back and validate file contents (hash/diff).

7.3.5 Practical mitigation: controlled writes + verification

RED's outcome is not merely “decompose further,” but to enforce a robust execution pathway:

1. Generate outline
2. Generate section chunks (bounded size / bounded symbol density)
3. Write each chunk via `write_file`
4. Verify file via `read_file` and compare against intended content

This transforms a fragile one-shot operation into an auditable sequence of primitives.

7.3.6 Why this example matters

This worked example demonstrates a key point: RED is especially valuable when failures arise from **interfaces** and **substrate constraints**, not from “incorrect reasoning.” A baseline plan often omits serialization boundaries because they are not salient at the narrative level. RED forces these boundaries into the decomposition and treats them as dependencies that must be verified.

More broadly, the example illustrates RED's general pattern:

- convert implicit assumptions into explicit dependencies,
- treat unverifiable dependencies as missing work,
- generate concrete mitigation tasks,
- and require verification steps that close the loop.

Empirical comparison table (observational)

Note: This table presents observational data from a single-user evaluation, not a controlled experiment. Results are illustrative of the failure mode and mitigation, not statistical claims.

Condition	Success rate	Typical failure	Notes
Baseline: one-shot <code>write_file</code> tool call (Markdown > ~1200 words, symbol-heavy)	< 10%	tool-call payload invalid / parse failure (serialization boundary breaks)	failures are catastrophic (no write)
Mitigation: WritePlanTool session protocol (begin → stream content → DONE → finalize)	100%*	Typical failure: <i>none observed (expected failure mode: finalize validation rejection)</i>	failures are localized and recoverable; avoids tool-call serialization boundary

*Observed across 137 write sessions over 6 days (single user, single system). Formal measurement with controlled conditions is future work.

7.4 RED for Post-Failure Root Cause Analysis

The preceding sections present RED as a **pre-execution audit**. This section extends RED to a complementary use case: **post-failure debugging**.

7.4.1 The Wrong Fix Problem

When a system fails, practitioners typically observe a symptom, form a hypothesis, and implement a fix. This workflow is efficient when intuitions are correct but costly when they are wrong:

- A wrong fix adds complexity without resolving the failure
- The real bug persists, obscured by additional code
- Debugging becomes harder as "fixes" accumulate

RED addresses this by treating root cause identification as a decomposition problem: **unverified hypotheses are treated as MISSING, not assumed correct**.

7.4.2 Protocol Adaptation

When applied to debugging, RED adapts as follows:

Step	Debugging Application
Scope	Define clear success criteria for stable behavior
Decompose	Break down baseline → hypothesis → verification into primitives

Step	Debugging Application
Verify	Run targeted tests before implementing any fix
Gate	Treat unverified hypotheses as MISSING

The critical gate: RED forbids jumping from hypothesis to implementation.

7.4.3 Case Study: LLM Orchestration System

We applied RED to debug an agentic workflow controller exhibiting intermittent failures. Traditional debugging had consumed over 14 hours without resolution; RED resolved the issue in approximately 2 hours.

Symptom: System hangs after certain responses. Multi-step operations fail partway through.

RED approach: We defined success criteria for stable system behavior, then decomposed the baseline phase into verification primitives:

ID	Action	Status
2.1	Document expected flow	VERIFIED_HAVE
2.2	Capture traces during failure	VERIFIED_HAVE
2.3	Query traces for anomalies	VERIFIED_HAVE

Trace analysis revealed **Bug #1**: state variables persisted across requests, causing misinterpretation of valid inputs.

The wrong hypothesis:

After fixing Bug #1, multi-step operations still failed. Initial hypothesis:

| "Pattern pollution—the LLM repeats previous output. We need context filtering."

We were prepared to implement filtering. However, RED required verification first.

Comparison test:

ID	Action	Status
3.1	Reproduce failure	VERIFIED_HAVE
3.2	Run comparison test (vary execution mode)	VERIFIED_HAVE
3.3	Confirm or refute hypothesis	VERIFIED_HAVE

Results:

- Interactive mode: **success**
- Non-interactive mode: **failure**

If pattern pollution were the cause, both modes would fail. The difference indicated a mode-specific bug.

Actual root cause: Non-interactive mode lacked session handling entirely—no completion detection, no finalization. The "pattern pollution" hypothesis was wrong.

7.4.4 Results

Metric	Ad-hoc (human + LLM)	RED-guided (human + LLM)
Time	14+ hours	~2 hours
Outcome	Unresolved	Resolved
Wrong fixes attempted	≥ 3	0
Bugs found	0	2

Key difference: Ad-hoc debugging with LLM assistance failed to identify the root cause after 14+ hours. The same human–LLM collaboration, guided by RED, resolved the issue in ~2 hours. RED was the differentiating factor.

While this is a single observational case, the magnitude of the difference—unresolved vs. resolved, and 7× faster when resolved—highlights the potential of structured, verification-gated debugging over ad-hoc approaches.

7.4.5 Why RED Prevented the Wrong Fix

The wrong fix (context filtering) would have:

1. Added complexity to context preparation
2. Been fragile (dependent on LLM patterns)
3. Left the actual bug unresolved
4. Made future debugging harder

RED prevented this by requiring a 5-minute comparison test that definitively refuted the hypothesis.

Key insight: RED's value is highest when intuitions are wrong—exactly when methodological discipline matters most.

7.4.6 Summary

RED extends from pre-execution auditing to post-failure debugging. The core mechanism—verification-gated decomposition—applies equally to "can this plan work?" and "why did this system fail?"

The key addition for debugging: **comparison testing** as a verification primitive—systematically varying conditions to isolate the actual root cause before implementing any fix.

Observation from single case study; formal evaluation is future work.

8. Conclusion

RED is a verification-gated decomposition protocol for surfacing hidden assumptions and missing fundamentals. By forcing decomposition to registered primitives and forbidding unverifiable “assumed okay” dependencies, RED converts high-level narratives into auditable tables and an actionable missing-fundamentals backlog. Case studies demonstrate how RED produces constraints, assumptions, and testable gaps rather than informal critique. Limitations are managed through scoping, safety gates, strict verification, and expert-in-the-loop validation.

References

- Darlow, L., Regan, C., Risi, S., Seely, J., & Jones, L. (2025). *Continuous Thought Machines*. arXiv:2505.05522.
- Fikes, R. E., & Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3-4), 189-208.
- Gawande, A. (2010). *The Checklist Manifesto: How to Get Things Right*. Metropolitan Books.
- Ghallab, M., Nau, D., & Traverso, P. (2004). *Automated Planning: Theory and Practice*. Morgan Kaufmann.
- McDermott, D., et al. (1998). PDDL—The Planning Domain Definition Language. Technical Report, Yale Center for Computational Vision and Control.
- Sacerdoti, E. D. (1974). Planning in a hierarchy of abstraction spaces. *Artificial Intelligence*, 5(2), 115-135.
- Stamatis, D. H. (2003). *Failure Mode and Effect Analysis: FMEA from Theory to Execution*. ASQ Quality Press.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. NeurIPS.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv:2210.03629*.
